

Embeddings:

Numeric representations of Words,
Documents, & Sentences

Large Language Models, Text Embeddings,
and Network Analysis

Sunbelt 2026, Morning Session



Thomas Bryan Smith
University of Mississippi

✉ tbsmit10@olemiss.edu

Ayan-Yue Gupta
University of Milan

 [thomasbryansmith/](https://github.com/thomasbryansmith/)

The Plan (for today)

- Workshop is split into two halves, each 3-hours:
 1. **Embeddings**
 - Building numeric representations of words, documents, and sentences.
 2. **Semantic Networks**
 - Mapping relationships between words, documents, sentences, and topics.
- *We will be using compact large language models to help generate embeddings—we will **not** be asking ChatGPT to do our work for us!*

Contents

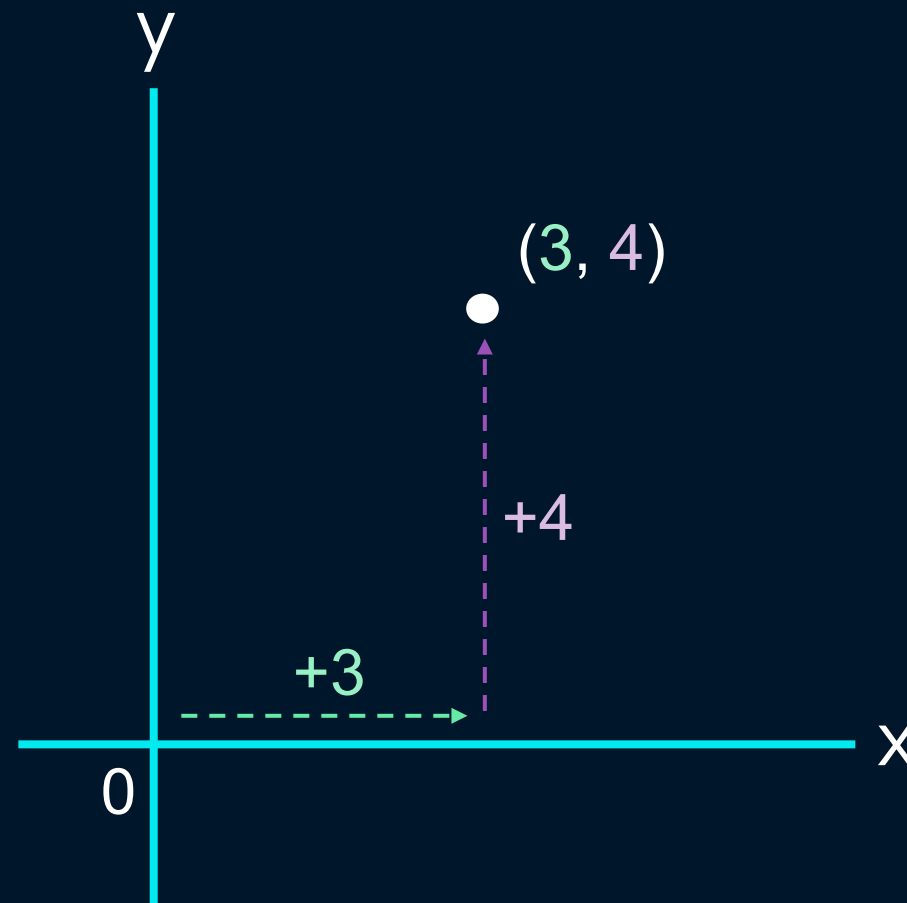
1. Introduction to 'Embeddings' (plural noun) and 'Embedding' (verb)
2. Word2Vec (and Doc2Vec)
3. Transformer-based models—primarily BERT
4. Practical Exercises
 - Introducing Python
 - Generating your own text embeddings

What are 'Embeddings'?

- Central feature of Large Language Models (LLMs) and Natural Language Processing (NLP).
- Emerged out of the need for computers to index (and retrieve or search for) documents.
- Salton (1968; 1975) proposed automating the process.
- His approach to this automation required that we develop methods for representing documents as a series of numbers—'Embeddings'.

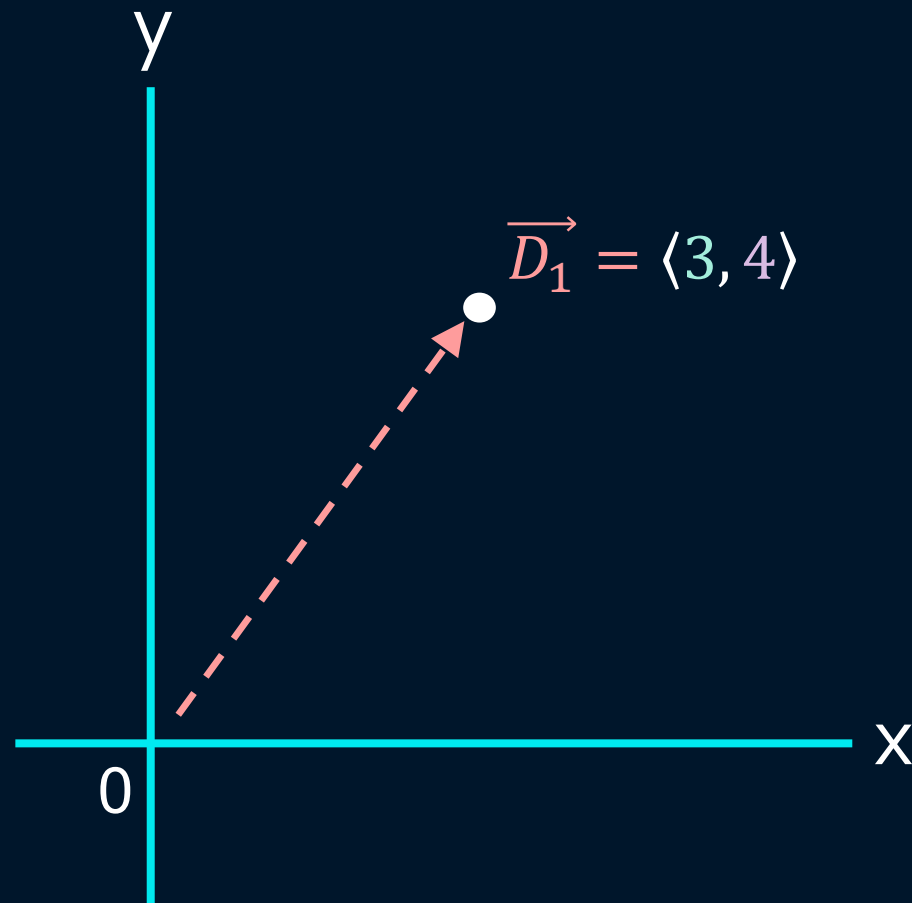
What are 'Embeddings'?

- Take a coordinate from a regular old scatterplot, e.g. $(x = 3, y = 4)$.
- This encodes the location in a 2-dimensional space wherein some given point of interest is **embedded**.
- So, when we refer to '**text embeddings**', we are simply referring to the **position in a space wherein a word, document, or sentence of interest is embedded**.



What are 'Embeddings'?

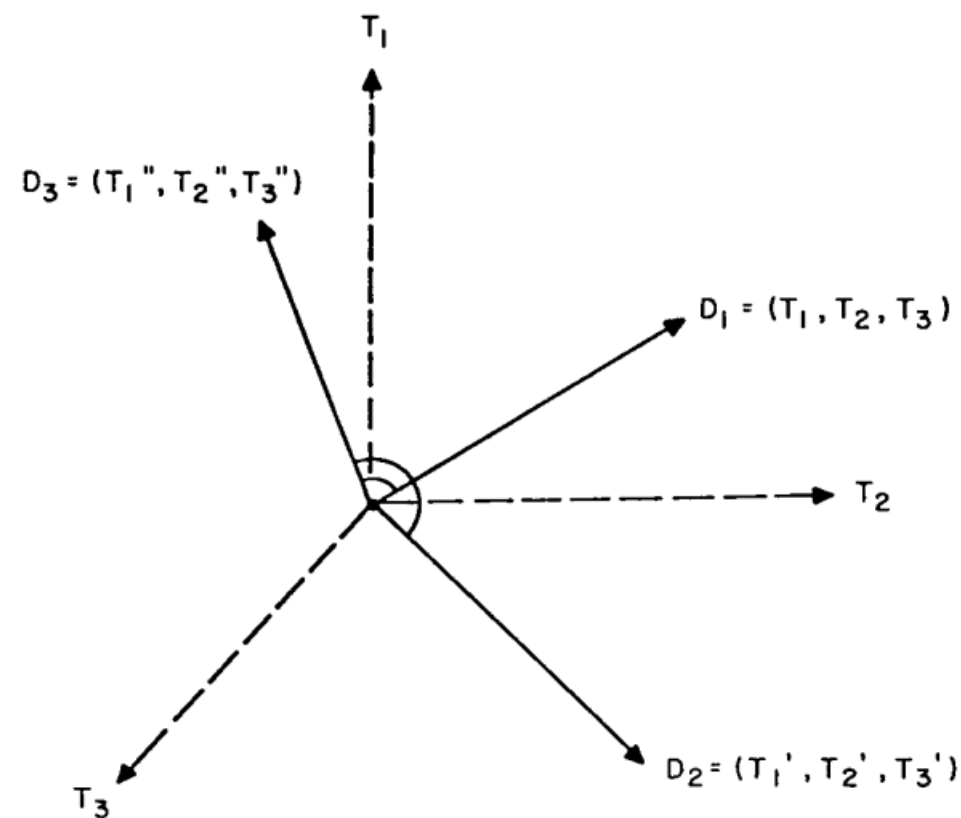
- These simple coordinates can also be described as a **position vector**.
 - **Position vectors** describe the direction in which you are pointing, starting out at the intercept, (0, 0).
- If we imagine that these numbers represent the location of a document, we can say that **Document 1 is at (3,4)**.



What are 'Embeddings'?

- This idea of embedding documents in k -dimensional space, then describing the documents in terms of their vector representation, was Salton's (1975) proposed solution to automated document indexing.

Fig. 1. Vector representation of document space.



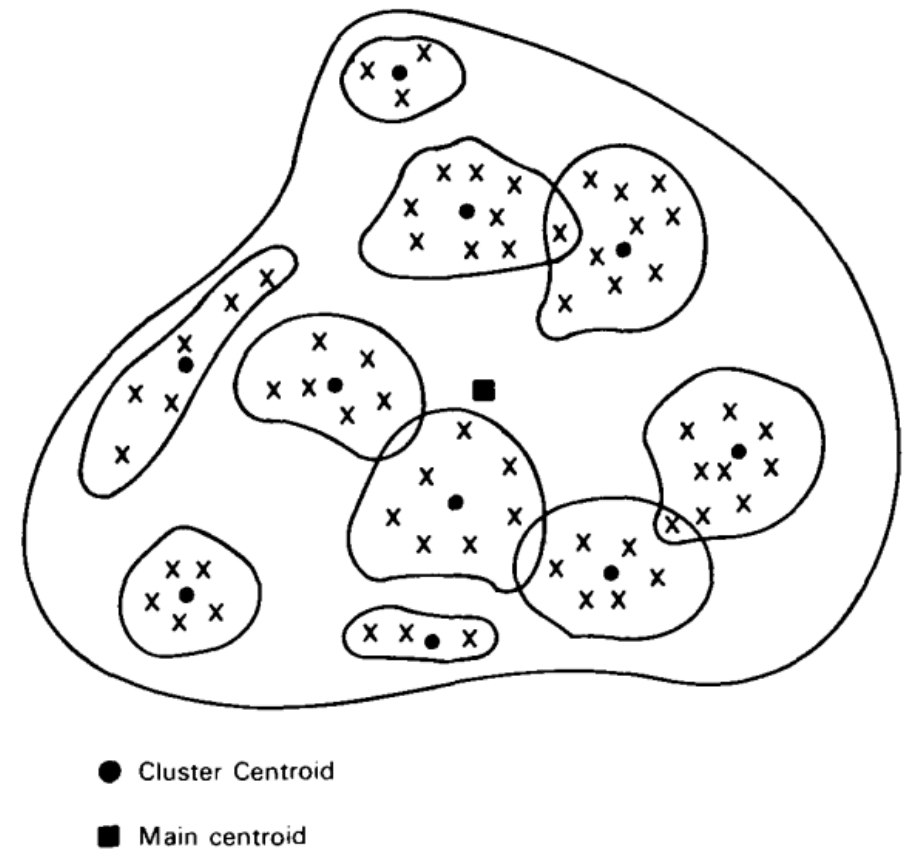
What are 'Embeddings'?

- When documents are represented in this way, it becomes possible to find the similarity between documents, or:

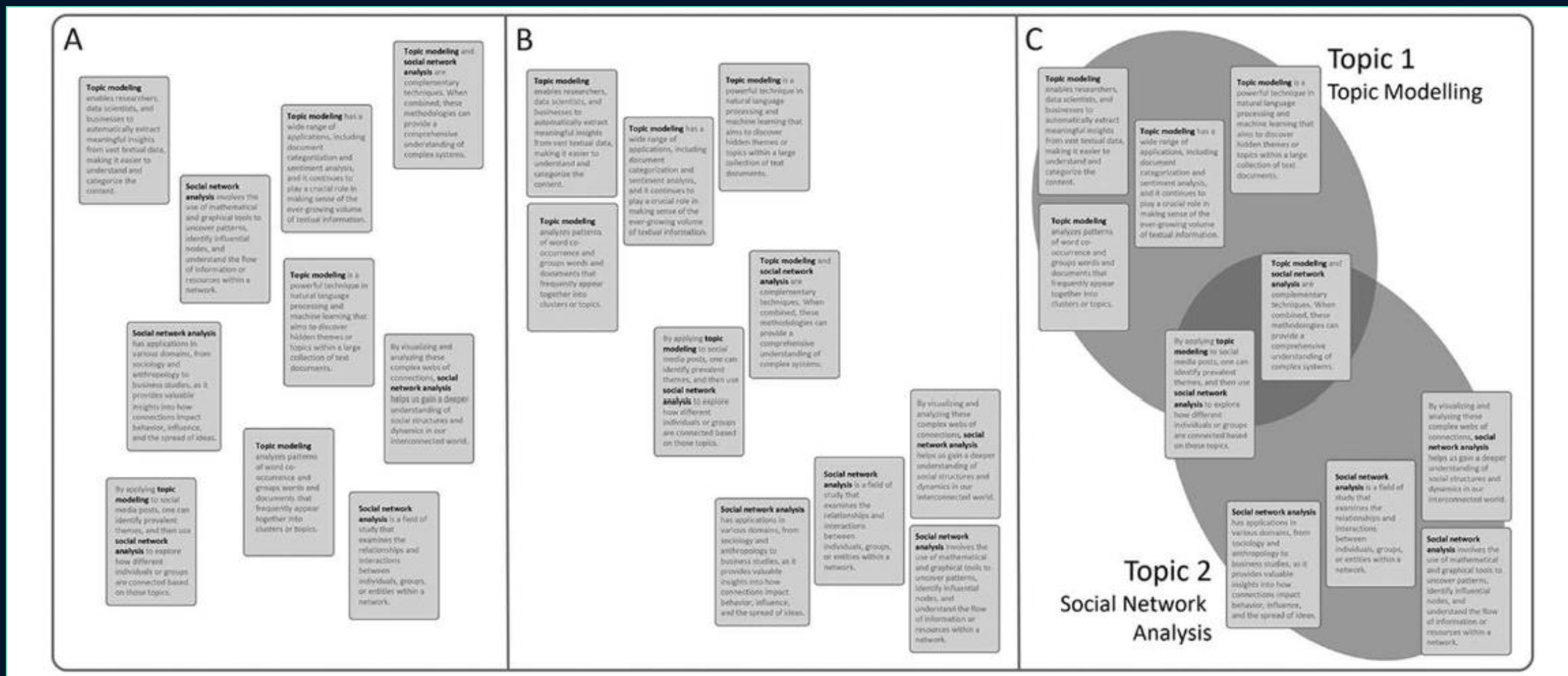
$$s(D_i, D_j).$$

- This similarity is integral in describing the clusters that emerge in the 'document space'—documents that are close together are related!

Fig. 4. Clustered document space.



What is 'Embedding'?



What is 'Embedding'?

- The question then becomes: what numbers should we use to represent the substance of documents, words, and sentences?
- Salton's (1975) original recommendation was a simple document-term matrix—which provides the foundation for many of the early topic models (Latent Semantic Indexing; Latent Dirichlet Allocation; etc.)
- He argued that the word frequencies under each document (i.e., the frequency of each word in the document) would serve as a numeric 'profile'—numeric representations of document that can be 'embedded' in space.

What is 'Embedding'?

A	Document-Term Matrix				
	analysis	model	network	social	topic
article 1	3	1	3	2	0
article 2	2	0	2	0	0
article 3	1	1	2	0	2
article 4	0	2	0	0	3
article 5	1	2	0	0	2

B	TF weighted Matrix				
	analysis	model	network	social	topic
article 1	0.333	0.111	0.333	0.222	0
article 2	0.5	0	0.5	0	0
article 3	0.167	0.167	0.333	0	0.333
article 4	0	0.4	0	0	0.6
article 5	0.2	0.4	0	0	0.4

C	TF-IDF weighted Matrix				
	analysis	model	network	social	topic
article 1	0.107	0.036	0.246	0.516	0
article 2	0.161	0	0.368	0	0
article 3	0.054	0.054	0.246	0	0.246
article 4	0	0.129	0	0	0.442
article 5	0.064	0.129	0	0	0.295

$$TF = \frac{\text{number of times the term appears in the document}}{\text{total number of terms in the document}}$$

$$IDF = \log \left(\frac{\text{number of documents in the corpus}}{\text{number of documents in the corpus that contain the term}} \right)$$

$$TF-IDF = TF * IDF$$

Word2Vec: Self-supervised word embedding

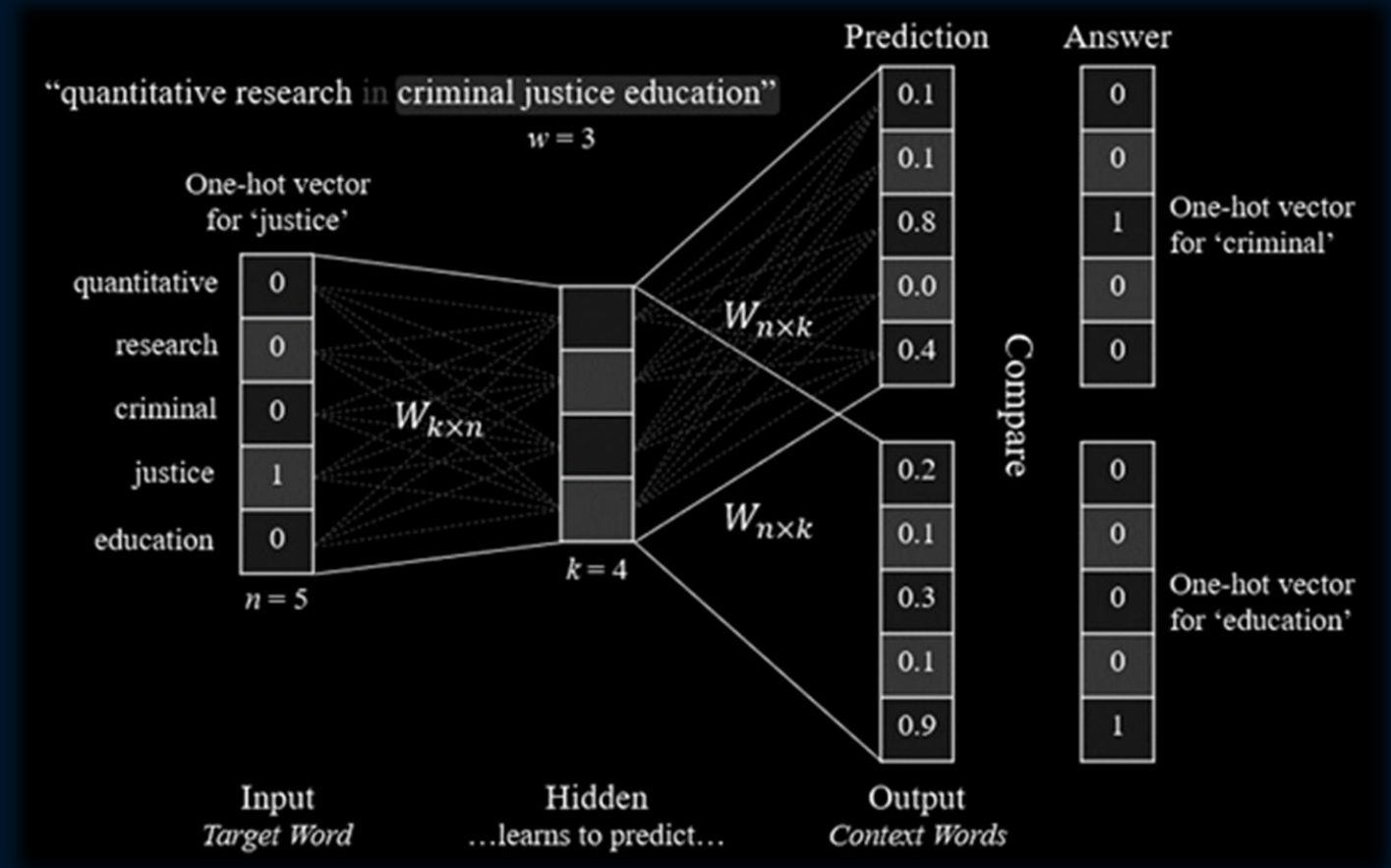
- Most of the early topic models—including Latent Semantic Indexing and the much more popular Latent Dirichlet Allocation—were developed by building on this ‘bag-of-words’ approach.
- However, the biggest limitation is that, by throwing words into a ‘bag’ (figuratively speaking), you disregard the meaning of word order.
 - “The detective told the judge that the suspect lied to the witness.”
 - “The judge told the witness that the detective lied to the suspect.”

Word2Vec: Self-supervised word embedding

- Word2Vec, developed by Tomas Mikolov et al. (2013a), prompted a paradigm shift in computational linguistics.
- Rather than treating words as 'indices in a vocabulary', Word2Vec uses neural networks to learn the numeric representations of words in a corpus.
- Two variants that are trained slightly differently:
 - Continuous Bag of Words (CBOW)—takes sets of context words, 'learns' to predict a hidden central word.
 - Skip-gram—takes a single target word, 'learns' to predict the surrounding context words.

Word2Vec: Self-supervised word embedding

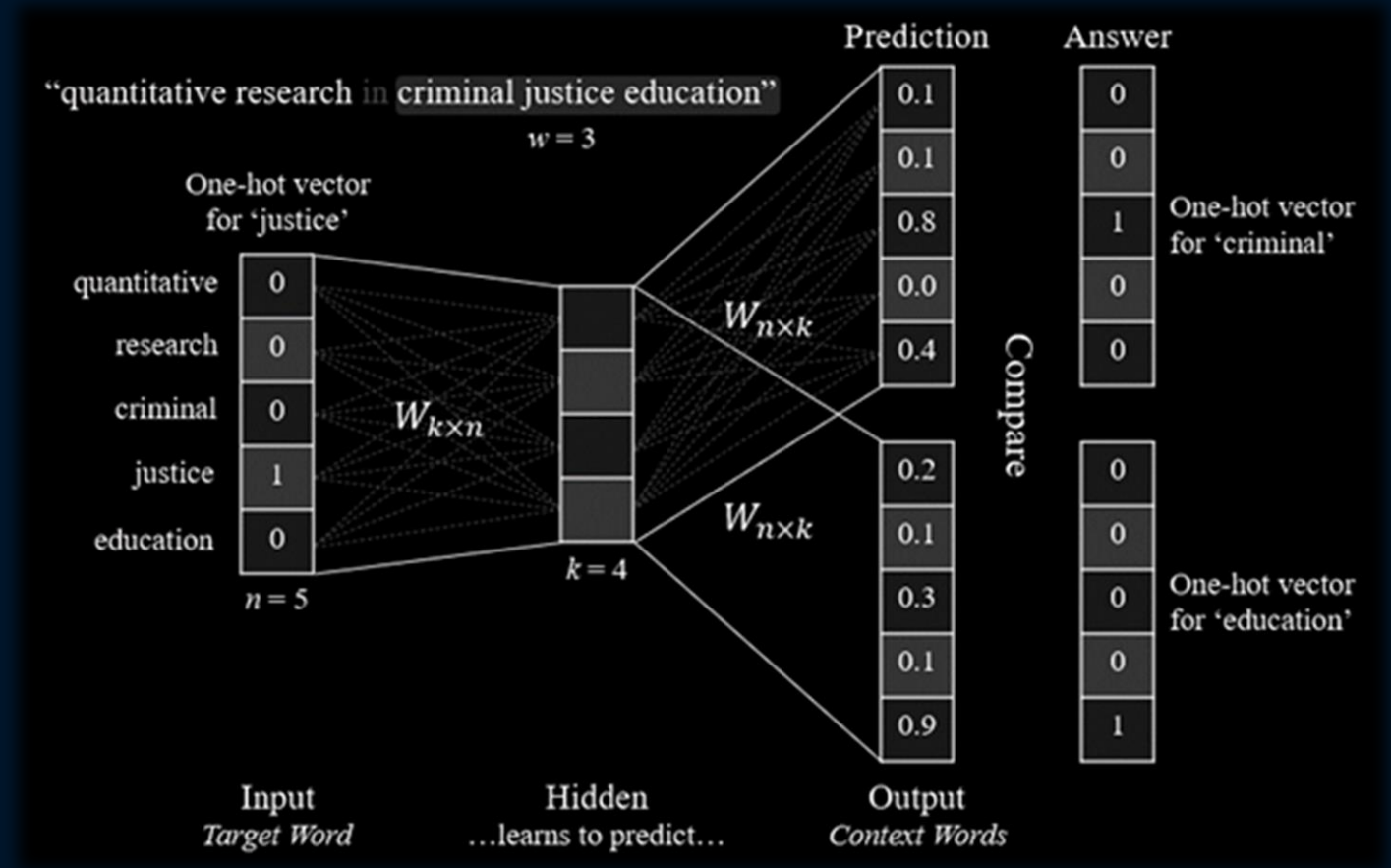
1. Input target word.
2. Weighted random sample of paths through the neural network leads to context word predictions.
3. Predicted word(s) correct?
 - **Yes:** Up-weight that path.
 - **No:** Down-weight that path.
4. Reiterate, updating weights with each iteration.



*Skip-gram model architecture

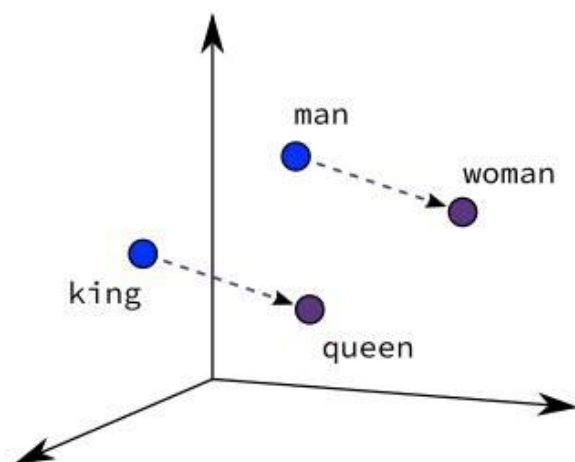
Word2Vec: Self-supervised word embedding

- Obviously, this process is much more complicated when expressed mathematically.
- However, the intuition matters more for this exercise.
- The benefit is that the numeric representations for words are learned in relation to sentence context.

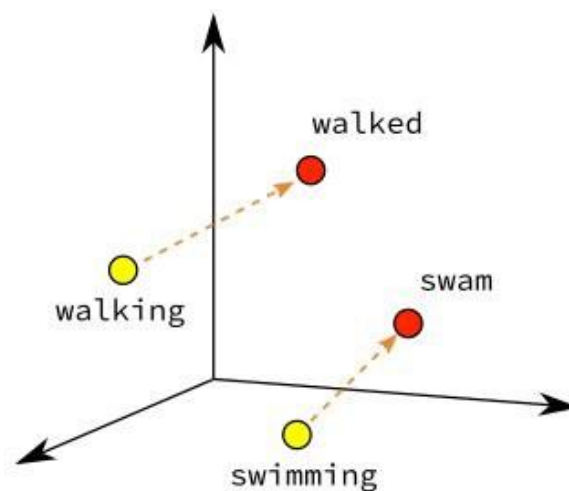


*Skip-gram model architecture

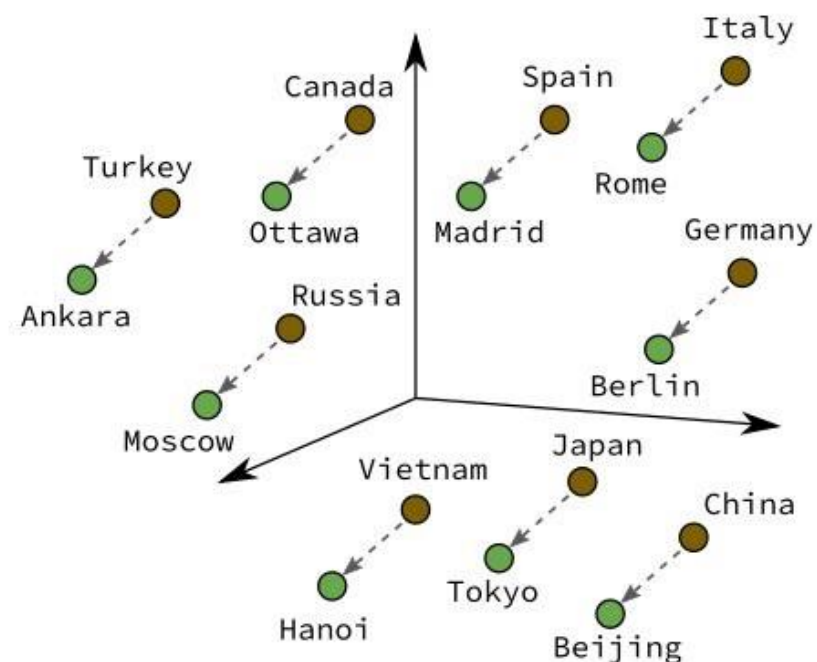
Word2Vec: Self-supervised word embedding



Male-Female



Verb Tense



Country-Capital

to sentence context.

<https://developers.google.com/machine-learning/crash-course/embeddings/translating-to-a-lower-dimensional-space>

Word2Vec: Hyperparameters

- Each approach to word2vec (skip-gram and DBOW) has **pros** and **cons**.
 - Skip-gram
 - Encodes semantic similarity rather than syntactic similarity (larger window size).
 - **Pros**: Able to learn meaning of rare words, better for learning word meaning.
 - **Cons**: Slower, computationally expensive, requires larger data.
 - Distributed Bag of Words (DBOW)
 - Encodes syntactic similarity rather than semantic similarity (smaller window size).
 - **Pros**: Faster, computationally efficient, smaller data, learns frequent words better.
 - **Cons**: Overfits frequent words, insufficient word detail.
- This makes model selection a 'hyperparameter' of sorts.

(Martel & Salamea, 2024)

Word2Vec: Hyperparameters

- **Vector Size**

- What length of vector is sufficient to capture word meaning in the corpus?
- Ranges between 100 and 300; 1,000 is extreme (and unnecessary!)
- Smaller vectors capture less information, easier to compute.
- Larger vectors capture more information, more difficult to compute.

(Mikolov et al., 2013a)

Word2Vec: Hyperparameters

- **Window (Width)**

- Window around a target word within which context words are identified.
- A window of '5' would imply 5 words on either side of the target word.
- Defaults are 5 for CBOW and 10 for skip-gram.
- 2-to-5 words is considered small; 10-to-20 words is considered large.
- Narrower is better for syntactic similarity; wider is better for semantic similarity.
- Additional notes:
 - Randomization sometimes implemented to calculate distance weights.
 - Check to see whether the implementation accommodates sentence boundaries!

(Brenndoerfer, 2025)

Word2Vec: Hyperparameters

- **Negative Sampling**

- In corpora containing an extreme number of unique words, Word2Vec can be extremely inefficient (because it needs to update all those words!)
- Many implementations will include an optional negative sampling hyperparameter which makes computation easier.
- Samples k incorrect output words for every correct pair—you define k .

(Mikolov et al., 2013b)

Word2Vec: Hyperparameters

- **Word Frequency Thresholding**

- Subsampling frequent words / omitting excessively rare words from the corpus based on a threshold—user defined.
- t is typically 10^{-5} , or 0.00001
 - Per Mikolov et al., 2013b—although, this may not hold for all implementations of word2vec!
- Some implementations just ask you for a simple threshold (including Python's *gensim*!)

$$P_t(w_i) = 1 - \sqrt{\frac{t}{f(w_i)}}$$

$$P_t(w_i) = \frac{f(w_i) - t}{f(w_i)} - \sqrt{\frac{t}{f(w_i)}}$$

(Mikolov et al., 2013b)

Word2Vec: Preprocessing is still important!

- **Phrase (N-gram) Identification**

- Pre-processing task.
- Identifies phrases by examining the frequency relative to the product of the words' individual frequencies.
- δ serves as a threshold for eliminating rare phrases.
- User needs to determine the threshold above which phrases are treated as a word.

$$\begin{aligned} & \text{score}(w_i, w_j) \\ &= \frac{\text{count}(w_i w_j) - \delta}{\text{count}(w_i) \times \text{count}(w_j)} \end{aligned}$$

network science



network_science

(Mikolov et al., 2013b)

Doc2Vec: A brief note about document embeddings

- Extension of word2vec that introduces a document/paragraph ID vector to the architecture of the word2vec neural network.
- The document/paragraph ID vector updates as word2vec trains, capturing the meaning of the document/paragraph in aggregate.
- Be aware that the weaknesses of word2vec (i.e., inability to handle polysemy) propagate to the document/paragraph vectors.
- Input is the same (your text data or 'corpus').
- Output is embeddings for each document or paragraph—not each word.

Doc2Vec: A brief note about document embeddings

- Like word2vec, doc2vec has two different architectures:
 - Distributed Memory (Skip-gram equivalent)
 - Distributed Bag of Words (DBOW; default)
- All the same caveats I discussed about word2vec also apply to doc2vec.
 - This is why DBOW is the default—the limitation of DBOW when modelling words (that it overfits frequent words) becomes a strength when modelling documents!
- Similarly, the hyperparameters are consistent.
- There is also top2vec, developed by Angelov (2020), which groups words and documents into topic clusters, and generates embeddings for the topics.

Bidirectional Encoder Representations from Transformers (BERT)

- Problems with word2vec and other early word embedding models:
 - Difficult to update pre-training embeddings; no “checkpointing”
 - Cannot handle polysemy—the state of one word being associated with multiple meanings (tends to ‘average out’ the meaning).
- To accommodate polysemy, we need dynamic word embeddings that represent the word in a context (e.g., sentence).
- BERT, a compact (early) LLM, solves both of these problems!

(Devlin et al., 2019)



Bidirectional Encoder Representations from Transformers (BERT)

- Comparing Word2Vec and BERT:
 - 'The bank approved my loan.'
 - 'She sat on the bank of the river.'
 - 'The bank charges high interest rates.'
 - 'He found himself lost in a fog bank.'
- Word2Vec → encodes 'bank' with a single embedding, the most popular tends to 'win out' when averaging across definitions.
- BERT → encodes 'bank' with four different embeddings, one for each sentence that includes the word.

(Devlin et al., 2019)



Bidirectional Encoder Representations from Transformers (BERT)

- Visualizing word2vec was easy, because it is a 'shallow' neural network with three layers (input, linear projection, output).
- BERT, on the other hand, has between 12 and 24 encoder layers, each containing a feed-forward and self-attention sub-layer.
 - For context, word2vec can be viewed as a *single* 'feed-forward' neural network *without* self-attention!
- The intuition is largely the same, with the caveat that a lot more 'things' happen to the data between input and output.

(Devlin et al., 2019)



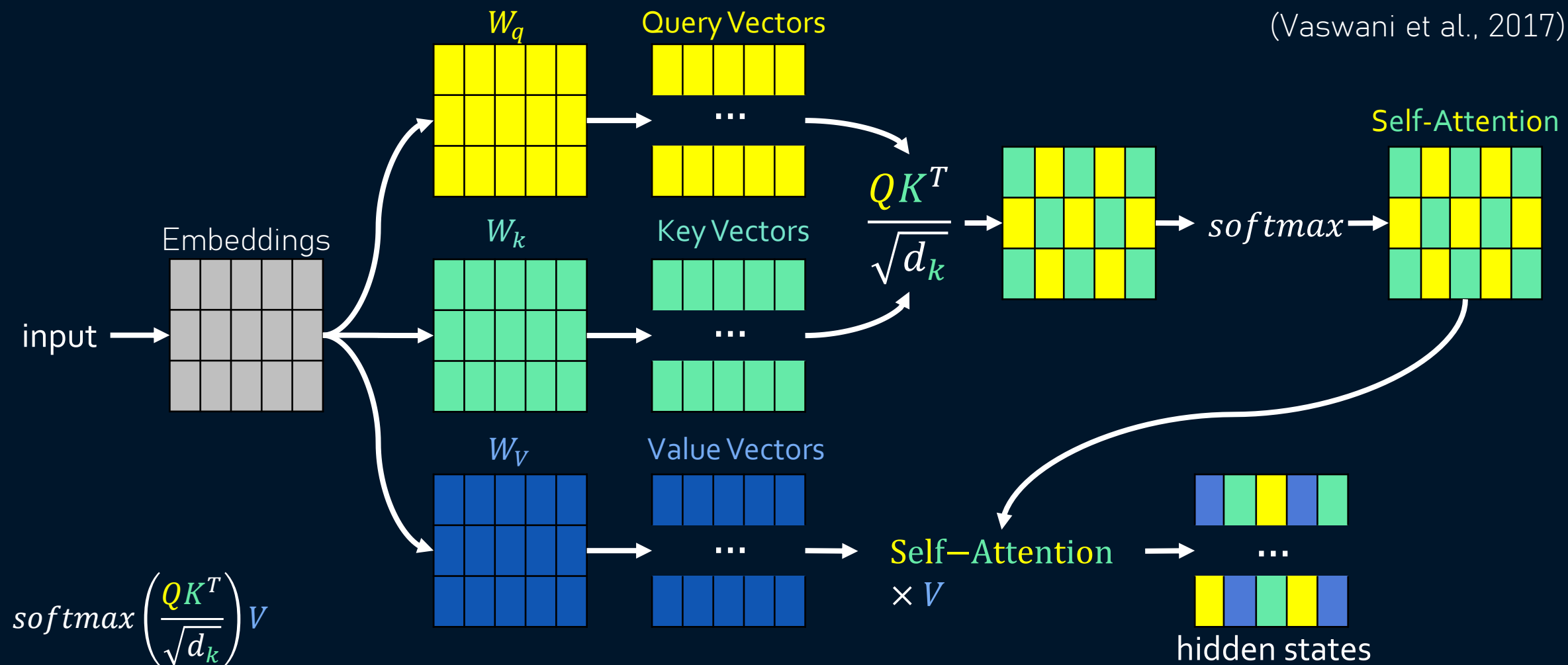
Self-Attention: Modeling polysemy

(Vaswani et al., 2017)

- Operates within Transformer-based models.
- Takes three things as input:
 1. **Query Vectors (Q)**: Focal 'target' word—same intuition as word2vec.
 2. **Key Vectors (K)**: Other words in the sequence (within the 'window').
 3. **Value Vectors (V)**: Data ('actual content') representing the words.
- Outputs an additional set of attention weights that represents 'context', which then updates (re-weights) the value, V .

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Self-Attention: Modeling polysemy



BERT: Hyperparameters

- BERT has a wide variety of applications (e.g., sentiment analysis, question classification, topic classification). So, there are not any hard or fast rules for tuning hyperparameters. (for recommended hyperparameters, see: Xipeng et al., 2020)
- Hyperparameters include:
 - **Learning Rate:** Rate that the model iteratively adjusts internal weights.
 - Usually between $2e-5$ and $5e-5$.
 - **Epochs:** The number of complete passes through the training data.
 - 2 to 4 for fine-tuning, 40 for training from scratch.
 - **Batch Size:** The number of text sequences iteratively processed.
 - 16 or 32—your device might catch fire if you try more than 32!
 - **Starting Checkpoint:** pre-trained model you start out with, then fine-tune.
 - Existing pre-trained models can be found on HuggingFace.

Bibliography

- Brenndoerfer, M. (2025). Skip-gram Model: Word2Vec Architecture and Word Embeddings. Available at: <https://mbrenndoerfer.com/writing/skip-gram-model-word2vec-word-embeddings>
- Martel, W., & Salamea, C. (2024, November). Evaluating Word Embedding Models in Ecuadorian Legal Texts: A Comparison of CBOW and Skip-Gram for Semantic Analysis. In *International Conference on Smart Technologies, Systems and Applications* (pp. 206-216). Cham: Springer Nature Switzerland.
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013a). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013b). Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems*, 26.
- Salton, G., & Lesk, M. E. (1968). Computer evaluation of indexing and text processing. *Journal of the ACM (JACM)*, 15(1), 8-36.
- Salton, G., Wong, A., & Yang, C. S. (1975). A vector space model for automatic indexing. *Communications of the ACM*, 18(11), 613-620.
- Smith, T. B. & Vacca, R. (2025) 18. Topic modeling and network analysis. In Nina Kolleck (Eds.), *Handbook of Social Network Analysis and Education* (pp.259-276). Edward Elgar. <https://doi.org/10.4337/9781035312191.00024>

Bibliography

- Angelov, D. (2020). Top2vec: Distributed representations of topics. *arXiv preprint arXiv:2008.09470*.
- BehnamGhader, P., Adlakha, V., Mosbach, M., Bahdanau, D., Chapados, N., & Reddy, S. (2024). Llm2vec: Large language models are secretly powerful text encoders. *arXiv preprint arXiv:2404.05961*.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019, June). Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)* (pp. 4171-4186).
- Grootendorst, M. (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure. *arXiv preprint arXiv:2203.05794*.
- Le, Q., & Mikolov, T. (2014, June). Distributed representations of sentences and documents. In *International conference on machine learning* (pp. 1188-1196). PMLR.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.